

Auditoría de la estructura del repositorio

Agente-RH

Alcance revisado

El repositorio `Agente-RH` se presenta como un **esqueleto operativo** para proyectos que utilizan un agente basado en Google Antigravity en un entorno Windows 11. Según el propio `README.md`, se trata de un repo *stack-agnóstico* que no incluye el código de la aplicación; su finalidad es proporcionar la estructura, las reglas del workspace, los flujos de trabajo y un **mini cerebro** (`brain/`) para mantener memoria operativa ¹.

Se examinaron los directorios y archivos listados en la política de estructura (`repo-structure.json`), las reglas del agente y los workflows del directorio `.agent`, la configuración (`.agent/config`), los scripts de automatización (`scripts/`) y las plantillas de stacks (Node API, Next.js, CLI en Python). También se revisaron los archivos actuales del “cerebro” para analizar su utilidad y el proceso de mantenimiento de memoria.

Lectura global del estado

- **Estructura y reglas.**

El directorio `.agent` define reglas de operación (`rules/`), workflows (`workflows/`), plantillas de informes (`templates/`) y configuración (`config/`).

La regla **Core Operating Rules** exige comunicaciones en español, soluciones concretas y consistencia en naming y rutas ². Las reglas de mantenimiento del cerebro y de higiene del repositorio clarifican cómo actualizar la memoria (`brain/`) y cómo evitar basura y duplicados ³ ⁴. Las reglas de **dispatch** y **context-budget** establecen criterios para seleccionar workflows y limitan el consumo de contexto, recomendando leer siempre la “capa rápida” del cerebro y expandir a la capa profunda sólo si hay incidentes, contradicciones o cambios de alcance ⁵ ⁶.

La configuración `repo-structure.json` define directorios y archivos obligatorios y es verificada por un script (`scripts/check-structure.ps1`). Otra configuración (`context-budget.json`) fija límites de líneas y caracteres de la capa rápida y parámetros para detectar duplicidad de texto ⁷.

- **Mini cerebro** (`brain/`).

Los ficheros de capa rápida (`now.md`, `current-state.md`, `stack.md`, `deep-summary.md`) están presentes pero contienen sólo información genérica de inicio de proyecto ⁸ ⁹ ¹⁰. La capa profunda (decisions, backlog, milestones, etc.) se halla vacía y carece de contenido real. Esto es coherente con que el repositorio es una plantilla, pero implica que el flujo de actualización de memoria se probará cuando se inicie un proyecto real.

El archivo `project-overview.md` especifica la visión y objetivos de tener un esqueleto operativo y evita detalles de la aplicación ¹¹, y `architecture.md` resume los bloques principales (directorios `.agent`, `brain`, `scripts` y `docs`) ¹².

- **Scripts de automatización.**

Los scripts de PowerShell proporcionan funciones para verificar estructura

(`check-structure.ps1`), limpiar el repo (`check-cleanliness.ps1`), gestionar la memoria (`update-brain-quick.ps1`, `update-brain-deep-summary.ps1`), instalar el hook de pre-commit, inicializar proyectos y generar documentación. Por ejemplo, `check-cleanliness.ps1` busca archivos temporales y duplicados y avisa si archivos críticos están vacíos ¹³.

Todos los scripts están escritos en PowerShell y asumen un entorno Windows; los manuales recomiendan ejecutarlos con `powershell -NoProfile -ExecutionPolicy Bypass` ¹⁴ ¹⁵.

- **Plantillas de stacks.**

Existen tres stacks prediseñados: una API en Node con Fastify ¹⁶, una aplicación Next.js ¹⁷ y un CLI de Python basado en argparse ¹⁸. Estos archivos utilizan cadenas de placeholder (p. ej., `__PROJECT_NAME__`, `__PROJECT_VISION__`) para ser reemplazadas cuando se genere un proyecto. No se incluyeron tests ni configuraciones de linting o formateo; cada stack contiene lo mínimo para arrancar.

- **Documentación.**

La carpeta `docs/` incluye guías de flujo operativo, eficiencia de prompts, higiene del repositorio, guía de CI opcional y troubleshooting de hooks. Los documentos recuerdan que la memoria del repo (`brain/`) es persistente mientras que la de AntigraVity es efímera ¹⁹. También recomiendan pedir una sola cosa por turno y limitar el contexto duplicado ²⁰.

Sin embargo, faltan guías de contribución (`CONTRIBUTING.md`), licencias y políticas de seguridad.

Hallazgos críticos

1. Dependencia exclusiva de PowerShell y Windows 11.

Todos los scripts clave (`bootstrap.ps1`, `run-checks.ps1`, `check-structure.ps1`, `install-hook.ps1`, etc.) están implementados en PowerShell y se ejecutan con `powershell / pwsh` ¹⁴. Esto impide usar la plantilla en macOS o Linux sin instalar PowerShell y adaptar los comandos. No existe una alternativa en Bash/Python ni instrucciones sobre cómo utilizar el proyecto en sistemas no Windows.

2. Ausencia de pruebas automáticas para las plantillas de código.

Los stacks de Node, Next.js y Python incluyen sólo un archivo de entrada y no vienen con suites de tests ni scripts de linting. La carpeta `tests/` contiene únicamente pruebas de Pester para scripts de PowerShell, pero no hay tests para los stacks de aplicación. Esto reduce la posibilidad de detectar errores en el código generado y rompe con la prioridad de mantener calidad técnica exigida en las reglas ².

3. Carencia de archivo de licencia y guía de contribución.

El repositorio no incluye un `LICENSE` ni un `CONTRIBUTING.md`. La falta de licencia deja ambiguo el uso legal del repositorio y la aportación de terceros, lo cual podría frenar la colaboración externa y violar políticas corporativas.

4. Plantillas de stack muy básicas sin consideraciones de seguridad y escalabilidad.

La API en Node expone el servidor en todas las interfaces con `host: "0.0.0.0"` y registra el logger en modo debug sin configuración de CORS ni validación de entrada ²¹. La aplicación Next.js no incluye configuración de Content Security Policy ni manejo de errores global, y la plantilla CLI de Python imprime directamente las variables sin gestión de logging ²². Estas

carencias pueden generar vulnerabilidades o malas prácticas si los desarrolladores reutilizan las plantillas sin ampliarlas.

5. Sistemas de control de contexto rígidos.

`context-budget.json` fija límites globales (máx. 200 líneas y 8 000 caracteres para la capa rápida, etc.) ⁷. Estos valores podrían resultar insuficientes para proyectos reales, y no se explica cómo ajustarlos en función del tamaño del equipo o del dominio. Además, `check-context-budget.ps1` no se revisó para ver su rigor, pero es posible que bloquee commits por exceder estos límites, dificultando la flexibilidad cuando el proyecto crezca.

6. Memoria vacía e inexistencia de protocolos de sincronización real.

Los archivos de `brain/` están en blanco o contienen texto de ejemplo. La regla de mantenimiento del cerebro obliga a actualizar la capa rápida después de cada cambio ³, pero no hay ejemplos prácticos sobre cómo registrar decisiones, hitos o backlog. Tampoco existe automatización para sincronizar la memoria con Antigravity; se confía en que el usuario ejecute manualmente los scripts. Esto puede generar inconsistencia entre conversaciones y memoria si se olvida sincronizar.

7. Escasa cobertura de seguridad y cumplimiento.

El repositorio no aborda temas como gestión de credenciales, cifrado de variables de entorno, escaneo de vulnerabilidades ni política de privacidad. Para un agente que podría manejar datos sensibles de recursos humanos, esta omisión es un riesgo importante.

Hallazgos importantes

1. Gran número de archivos requeridos.

`repo-structure.json` especifica más de cien archivos obligatorios ²³. Esto ayuda a mantener un estándar, pero también incrementa la complejidad inicial. Algunas plantillas (`bug-finding-template.md`, `user-instruction-template.md`) están vacías; se deberían llenar con ejemplos o eliminar si no aportan valor.

2. Falta de herramientas de formateo y linting.

No se incluyen configuraciones de ESLint/Prettier para Node o Next, ni `flake8/black` para Python. La ausencia de un `.editorconfig` también puede generar inconsistencias.

3. Duplicidad de conceptos entre reglas y documentación.

Varias reglas (core, brain-maintenance, repo-hygiene) repiten recomendaciones similares sobre no duplicar documentos, actualizar capa rápida y usar scripts ² ²⁴. Esta repetición puede dificultar entender qué documento es la fuente de verdad. Consolidar o referenciar secciones evitaría mantener instrucciones redundantes.

4. Hooks opcionales y CI desactivado.

La guía de CI indica que las comprobaciones remotas son opcionales y que el hook pre-commit debe instalarse manualmente ²⁵. Esta laxitud va en contra de las reglas de calidad que exigen verificar la estructura y el contexto antes de cada commit ²⁶.

5. Falta de internacionalización en los flujos de trabajo.

Todos los workflows y documentación están en español (lo que cumple con la prioridad de hablar siempre en español ²⁷), pero no se contemplan proyectos que necesiten otros idiomas.

Definir un mecanismo de traducción o una política de idiomas podría ser útil si el proyecto se abre a colaboradores no hispanohablantes.

6. Implementación parcial del catálogo de skills.

El archivo `CATALOG.md` aparece en la configuración como obligatorio, pero no se incluye en la búsqueda. Además, `skills-available.md` indica que el catálogo se definirá mediante un workflow específico ²⁸; no obstante, no se documenta cómo extender `skills-sources.json` ni cómo manejar dependencias de skills.

Detalles finos y micro-mejoras

- Añadir un **archivo** `.editorconfig` para unificar estilo (indentación, fin de línea) en todos los lenguajes.
- En el stack de Node, reemplazar `process.exit(1)` por `app.close()` para permitir cierre limpio y facilitar pruebas unitarias ²¹.
- En la plantilla Next.js, extraer estilos a un fichero CSS modular en lugar de definirlos en línea ²⁹; esto favorece la escalabilidad del diseño.
- En la plantilla CLI de Python, utilizar el módulo `logging` en vez de `print` para permitir distintos niveles de log y facilitar la integración con otros sistemas ²².
- Documentar en los scripts el uso de `Resolve-Path` y capturar excepciones más específicas para mejorar la legibilidad y la robustez (por ejemplo, sustituyendo `throw` con mensajes de error más detallados).
- Incorporar un archivo `requirements.txt` para la plantilla Python y un `package.json` con scripts de test y linting para las plantillas Node/Next.
- Crear alias o scripts equivalentes en **Bash** para cada script de PowerShell, permitiendo que los desarrolladores de macOS/Linux utilicen el repo sin instalar PowerShell.
- Definir variables de entorno en las plantillas y documentar su configuración (por ejemplo, `PORT`, `NODE_ENV`, `LOG_LEVEL` para Node).

Incoherencias detectadas

- **Desfase entre la política de calidad y la implementación.** Las reglas enfatizan la necesidad de evitar duplicidad y de mantener memoria actualizada ², pero el repositorio incluye múltiples documentos que repiten las mismas instrucciones.
- **Incompatibilidad con la visión stack-agnóstica.** El README recalca que el repo es stack-agnóstico ³⁰; sin embargo, se incluyen plantillas concretas para Node/Next/Python sin plantillas equivalentes para otros stacks mencionados (Go, PHP). Esto puede generar expectativas de soporte que el repositorio no cumple ³¹.
- **Falta de sincronía entre scripts y documentación.** El documento `operating-flow-win11.md` indica que hay que ejecutar `scripts/run-checks.ps1` tras los cambios ³²; sin embargo, la regla de pre-commit se instala manualmente y puede no estar en ejecución.

Deuda técnica y riesgos

- **Portabilidad limitada:** La dependencia en PowerShell crea deuda futura cuando se intente usar la plantilla en entornos distintos (macOS, Linux, contenedores). Migrar o duplicar los scripts en un lenguaje multiplataforma como **Python** o **Bash** evitará este obstáculo.

- **Escalado de contexto:** A medida que el proyecto crezca, los límites de `context-budget.json` pueden ser insuficientes, obligando a reconfigurarlos. No existe tooling para recalcular automáticamente estos límites según el tamaño del repo.
- **Gestión de skills:** La falta de un catálogo de skills real y de documentación sobre cómo instalarlos dificulta integrar funcionalidades de IA adicionales.
- **Ausencia de prácticas DevSecOps:** No hay integraciones con herramientas de análisis de seguridad, ni flujos de CI para ejecutar linters y pruebas. Esto aumentará la deuda cuando el código empiece a crecer.

Oportunidades de mejora profunda

1. **Crear versiones multiplataforma de los scripts** (Bash/Python) y actualizar la documentación para describir cómo trabajar en Linux o macOS.
2. **Incorporar un sistema de pruebas automatizadas** para las plantillas de stacks (por ejemplo, Jest para Node/Next, Pytest para Python), así como linters (ESLint, Ruff) y formatters.
3. **Implementar un pipeline CI/CD básico** (GitHub Actions) basado en `scripts/run-checks.ps1` y en las nuevas pruebas, y documentar cómo habilitar la protección de rama.
4. **Añadir archivos de licencia y contribución**, así como un `SECURITY.md` que explique cómo reportar vulnerabilidades.
5. **Expandir el catálogo de stacks o modularizarlo** para incluir ejemplos en Go, PHP u otros lenguajes, alineado con la declaración de “stack-agnóstico”.
6. **Desarrollar scripts de sincronización automática** entre Antigraity y `brain/` para reducir el riesgo de que se pierdan decisiones o conclusiones al cerrar una sesión ¹⁹.
7. **Crear plantillas de registro de decisiones, hitos y backlog** con ejemplos claros, de forma que los usuarios puedan seguir un patrón consistente.
8. **Mejorar la seguridad de las plantillas** añadiendo configuraciones de CORS, sanitización de entradas y separación de variables de entorno.

Propuesta priorizada

Nivel de prioridad	Acción propuesta	Impacto esperado
Crítico	Implementar scripts multiplataforma o portar las utilidades a Python para permitir uso en macOS/Linux; ajustar la documentación en consecuencia.	Amplía la audiencia y reduce la dependencia de Windows, disminuyendo la deuda técnica y el riesgo de bloqueo.
Crítico	Añadir licencia, guía de contribución y archivo <code>SECURITY.md</code> .	Define marco legal y procedimiento de aportación; mejora la transparencia y conformidad.
Crítico	Integrar suites de pruebas y linters para cada stack y preparar un workflow de CI que ejecute <code>run-checks</code> y los tests.	Mejora la calidad de las plantillas y previene la propagación de errores.
Alta	Revisar y modularizar la configuración de <code>context-budget.json</code> , proporcionando instrucciones para ajustarla según el tamaño del proyecto.	Evita bloqueos innecesarios en commits y mantiene la eficiencia del contexto.

Nivel de prioridad	Acción propuesta	Impacto esperado
Alta	Completar la documentación de <code>brain/</code> con ejemplos de decisiones, hitos y backlog, y automatizar la sincronización con Antigravity mediante scripts programados.	Facilita que los usuarios sigan el flujo de memoria correctamente y reduce riesgo de inconsistencia.
Media	Mejorar las plantillas de Node y Next añadiendo manejo de errores, configuración de CORS y variables de entorno; migrar los estilos inline de Next a CSS externo.	Aumenta la robustez y seguridad de los proyectos generados.
Media	Simplificar las reglas fusionando recomendaciones duplicadas y referenciando una fuente de verdad única.	Reduce el ruido y facilita comprender las reglas del repositorio.
Baja	Añadir <code>.editorconfig</code> y configuraciones de formato para unificar estilo de código.	Mejora la legibilidad y consistencia, aunque no bloquea el avance.
Baja	Incluir plantillas de stacks adicionales o proporcionar guías para crear nuevos stacks.	Amplía la versatilidad pero no es urgente para comenzar un proyecto.

Plan por fases

- Fase 1 – Legalidad y configuración básica (inmediato):** añadir `LICENSE`, `CONTRIBUTING.md` y `SECURITY.md`; introducir `.editorconfig` y configurar linters mínimos.
- Fase 2 – Portabilidad y CI (corto plazo):** portar scripts a Bash/Python, actualizar documentación y crear workflows de CI en GitHub Actions que ejecuten los chequeos y tests.
- Fase 3 – Pruebas y seguridad (medio plazo):** añadir suites de pruebas unitarias para cada stack, mejorar la seguridad de las plantillas (CORS, manejo de errores) y habilitar escáneres de vulnerabilidades.
- Fase 4 – Amplificación del catálogo y automatización del cerebro (largo plazo):** crear más stacks o herramientas para generar stacks customizados, completar plantillas de decisiones/backlog y desarrollar scripts que sincronicen la memoria con Antigravity automáticamente.

Conclusión

El repositorio `Agente-RH` constituye una base sólida para iniciar proyectos asistidos por agentes en Antigravity; ofrece una estructura ordenada, reglas claras y herramientas para mantener el orden y la memoria operativa. Sin embargo, su dependencia de PowerShell, la falta de pruebas y la ausencia de archivos legales limitan su utilidad en entornos diversos y pueden generar deuda técnica. Abordar las mejoras aquí recomendadas —especialmente en portabilidad, pruebas y documentación— permitirá que la plantilla cumpla su objetivo de ser stack-agnóstica, escalable y profesional.

- 2 **core.md**
<https://github.com/ricardohg1998-create/Agente-RH/blob/master/.agent/rules/core.md>
- 3 24 **brain-maintenance.md**
<https://github.com/ricardohg1998-create/Agente-RH/blob/master/.agent/rules/brain-maintenance.md>
- 4 26 **repo-hygiene.md**
<https://github.com/ricardohg1998-create/Agente-RH/blob/master/.agent/rules/repo-hygiene.md>
- 5 **workflow-dispatch.md**
<https://github.com/ricardohg1998-create/Agente-RH/blob/master/.agent/rules/workflow-dispatch.md>
- 6 **context-budget.md**
<https://github.com/ricardohg1998-create/Agente-RH/blob/master/.agent/rules/context-budget.md>
- 7 **context-budget.json**
<https://github.com/ricardohg1998-create/Agente-RH/blob/master/.agent/config/context-budget.json>
- 8 **current-state.md**
<https://github.com/ricardohg1998-create/Agente-RH/blob/master/brain/current-state.md>
- 9 **stack.md**
<https://github.com/ricardohg1998-create/Agente-RH/blob/master/brain/stack.md>
- 10 **deep-summary.md**
<https://github.com/ricardohg1998-create/Agente-RH/blob/master/brain/deep-summary.md>
- 11 31 **project-overview.md**
<https://github.com/ricardohg1998-create/Agente-RH/blob/master/brain/project-overview.md>
- 12 **architecture.md**
<https://github.com/ricardohg1998-create/Agente-RH/blob/master/brain/architecture.md>
- 13 **check-cleanliness.ps1**
<https://github.com/ricardohg1998-create/Agente-RH/blob/master/scripts/check-cleanliness.ps1>
- 15 19 32 **operating-flow-win11.md**
<https://github.com/ricardohg1998-create/Agente-RH/blob/master/docs/operating-flow-win11.md>
- 16 21 **index.ts**
<https://github.com/ricardohg1998-create/Agente-RH/blob/master/tools/stacks/node-api/template/src/index.ts>
- 17 29 **layout.tsx**
<https://github.com/ricardohg1998-create/Agente-RH/blob/master/tools/stacks/next-app/template/app/layout.tsx>
- 18 22 **cli.py**
https://github.com/ricardohg1998-create/Agente-RH/blob/master/tools/stacks/python-cli/template/src/_PYTHON_PACKAGE_/cli.py
- 20 **prompt-efficiency.md**
<https://github.com/ricardohg1998-create/Agente-RH/blob/master/docs/prompt-efficiency.md>
- 23 **repo-structure.json**
<https://github.com/ricardohg1998-create/Agente-RH/blob/master/.agent/config/repo-structure.json>
- 25 **ci-optional.md**
<https://github.com/ricardohg1998-create/Agente-RH/blob/master/docs/ci-optional.md>
- 27 **AGENTS.md**
<https://github.com/ricardohg1998-create/Agente-RH/blob/master/AGENTS.md>

28 **skills-available.md**

<https://github.com/ricardohg1998-create/Agente-RH/blob/master/brain/skills-available.md>